

# Natural Language Processing

100 Questions: Pre-processing, BoW, TF-IDF, Naïve Bayes, Evaluation

GARP RAI Exam Preparation

**QUESTION: What is Natural Language Processing (NLP)?**

**OPTIONS:**

- ❶ A technique for processing numerical data
- ❷ An aspect of machine learning concerned with understanding and analyzing human language, both written and spoken
- ❸ A method for image recognition
- ❹ A database management technique

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Definition:** NLP is the intersection of computer science, artificial intelligence, and linguistics
- **Core purpose:** Enable computers to understand, interpret, and generate human language
- **Key challenge:** Most information exists in unstructured, free text format
- **Alternative names:** Content analysis, text mining, computational linguistics
- **Why it's important:**
  - Unstructured text is the most abundant form of data
  - Manual analysis is slow, expensive, and inconsistent
  - NLP enables automated analysis at scale

**EXAM TRAP:** NLP is about **understanding human language** - not just processing text!

## Q2: NLP vs Numerical Data - Tutorial

**QUESTION:** Why is NLP more challenging than modeling purely numerical data?

**OPTIONS:**

- ① Numerical data is always larger
- ② Textual data is often noisy with errors and ambiguities
- ③ NLP requires more computing power
- ④ Numerical data is more interesting

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Key challenges in NLP:**

- ① **Ambiguity:** Same words have different meanings (polysemy)
- ② **Context dependence:** Meaning depends on surrounding text
- ③ **Noise:** Spelling errors, abbreviations, colloquialisms
- ④ **Informal language:** Social media, text messages
- ⑤ **Sarcasm and tone:** Hard for machines to detect

• **Examples:**

- "u r gr8" vs "you are great"
- "lol" can mean different things in different contexts
- "That's just great!" - could be sincere or sarcastic

• **Impact:**

- NLP may fail to fully capture document essence
- Power and flexibility are improving rapidly
- Advances in computational capacity help

**EXAM TRAP:** NLP challenges include **ambiguity, noise, and context** - not just technical issues!

## Q3: NLP Applications in Finance - Tutorial

**QUESTION:** Which of the following is an NLP application in finance?

**OPTIONS:**

- ❶ Fraud detection using SEC filings
- ❷ Customer service call routing
- ❸ Sentiment analysis from social media
- ❹ All of the above

**ANSWER: D**

**DETAILED TUTORIAL:**

• **Key NLP applications in finance:**

❶ **Fraud detection:**

- SEC used NLP to detect accounting fraud
- Analyze financial reports for inconsistencies

❷ **Call routing:**

- Keyword recognition for customer inquiries
- Automated triage to appropriate departments

❸ **Sentiment analysis:**

- Social media monitoring
- Market reaction assessment
- Academic studies on asset prices and sentiment

❹ **Readability assessment:**

- Fog index measurement
- Plain English compliance

❺ **Document similarity:**

- Assessing new information content
- Tracking disclosure changes

**EXAM TRAP:** NLP has many financial applications - fraud detection, sentiment, customer service!

## Q4: NLP vs Manual Reading - Tutorial

**QUESTION:** What are the main benefits of NLP over manual document reading?

**OPTIONS:**

- ❶ NLP is always more accurate than humans
- ❷ Superior speed, no scope to miss aspects, and consistent application
- ❸ NLP is cheaper than human labor
- ❹ NLP doesn't make mistakes

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Key advantages of NLP:**

❶ **Speed:**

- Processes thousands of documents quickly
- Far faster than human reading
- Enables real-time analysis

❷ **Completeness:**

- No scope to miss any aspects
- Provided they are built into the design
- Consistent across all documents

❸ **Consistency:**

- All documents considered identically
- No scope for biases
- No inconsistencies in application

• **Limitations:**

- NLP may miss nuance
- Ambiguity remains challenging
- Requires careful design

**EXAM TRAP:** NLP advantages = **speed, completeness, and consistency!**



## Q6: JPMorgan COiN Platform - Tutorial

**QUESTION:** What did JPMorgan Chase's COiN platform achieve?

**OPTIONS:**

- ❶ It made no significant impact
- ❷ Significantly reduced time required to review legal documents
- ❸ It replaced all human reviewers
- ❹ It only worked for English documents

**ANSWER: B**

**DETAILED TUTORIAL:**

- **COiN (Contract Intelligence) Platform:**

- Automated review of legal documents
- Focus on commercial loan agreements

- **Challenges addressed:**

- Manual review was time-consuming
- Manual review was prone to errors
- Extracting information from unstructured text

- **Outcomes:**

- Significantly reduced review time
- Improved consistency in information extraction
- Improved accuracy in information extraction

- **Training:**

- NLP models trained on annotated legal documents
- Performance improved over time

**EXAM TRAP:** COiN reduced legal document review time significantly!

## Q7: Other NLP Use Cases - Tutorial

**QUESTION:** Which institutions use NLP for fraud detection?

**OPTIONS:**

- ❶ Only JPMorgan Chase
- ❷ HSBC uses NLP for fraud detection
- ❸ Only the SEC uses NLP
- ❹ No financial institutions use NLP for fraud detection

**ANSWER: B**

**DETAILED TUTORIAL:**

- **NLP use cases in banking:**
  - ❶ **Fraud detection:**
    - HSBC uses NLP for fraud detection
    - Analyze patterns in financial transactions
  - ❷ **Customer service:**
    - Bank of America (BoA) chatbots
    - Automated customer interaction
  - ❸ **Market sentiment:**
    - BlackRock uses NLP for sentiment analysis
    - Analyze market sentiment from news and social media
- **Early examples:**
  - SEC used NLP to detect accounting fraud
  - Pioneered the use of NLP in finance

**EXAM TRAP:** HSBC = **fraud detection**, BoA = **chatbots**, BlackRock = **sentiment analysis**!



## Q8: Corpus Definition - Tutorial

**QUESTION:** What is a corpus in the context of NLP?

**OPTIONS:**

- ① A single document
- ② The collection of all documents retrieved for analysis
- ③ A type of machine learning algorithm
- ④ A data pre-processing technique

**ANSWER:** B

**DETAILED TUTORIAL:**

• **Definition:**

- From Latin "corpus" meaning "body"
- Collection of all documents available for analysis
- May include hundreds or thousands of documents

• **Characteristics:**

- All documents in the corpus are processed together
- Vocabulary built from the corpus
- Analysis results apply to the corpus

• **Examples:**

- All 10-K filings of S&P 500 companies
- All tweets about a specific stock
- All customer service emails

**EXAM TRAP:** Corpus = collection of all documents - not a single document!

# Q9: NLP Pre-processing Importance - Tutorial

**QUESTION:** Why is data pre-processing important in NLP?

**OPTIONS:**

- ❶ It's not important
- ❷ To convert raw text into a format suitable for analysis
- ❸ To make documents shorter
- ❹ To remove all words

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Purpose of pre-processing:**
  - ❶ Convert text to machine-readable format
  - ❷ Remove non-textual information (HTML, tags)
  - ❸ Standardize the text
  - ❹ Reduce noise and improve accuracy
- **Initial cleaning steps:**
  - Extract text from PDFs
  - Remove HTML code
  - Handle emojis (convert to text or numerical values)
  - Use tools like demoji module in Python
- **Why quality matters:**
  - Newswires: formal, error-free
  - Social media: spelling errors, colloquialisms, abbreviations
  - Editing needed for most common issues

**EXAM TRAP:** Pre-processing converts **raw text to analyzable format!**

# Q10: Document Similarity - Tutorial

**QUESTION:** What is a practical use of document similarity in finance?

**OPTIONS:**

- ❶ To make documents longer
- ❷ To assess whether a news article constitutes new information
- ❸ To delete duplicate documents
- ❹ To remove all documents

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Document similarity applications:**

❶ **New information assessment:**

- Determine if news article provides new information
- Avoid redundant analysis
- Focus on novel content

❷ **Disclosure tracking:**

- Assess if tone of accounting disclosures has changed
- Monitor consistency in communications
- Detect significant changes

• **How it works:**

- Compare document vectors
- Compute similarity scores
- Identify duplicates or near-duplicates
- Track evolution of content

**EXAM TRAP:** Document similarity helps assess **new information content!**

# Q11: Tokenization Definition - Tutorial

**QUESTION:** What is tokenization in NLP pre-processing?

**OPTIONS:**

- ❶ Removing all punctuation from text
- ❷ Separating a document into sentences and then into words
- ❸ Converting all text to lowercase
- ❹ Removing stop words

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Tokenization process:**

❶ **Sentence tokenization:**

- Split document at periods, question marks, exclamation marks
- Creates sentence boundaries

❷ **Word tokenization:**

- Split sentences into words
- Remove punctuation, spacing, special symbols
- Convert to lowercase

• **Importance:**

- Creates individual text units for analysis
- Foundation for all subsequent NLP steps
- Enables counting and vectorization

**EXAM TRAP:** Tokenization = **splitting** into sentences and words!

# Q12: Stop Words Removal - Tutorial

**QUESTION:** What are stop words and why are they removed?

**OPTIONS:**

- ➊ Important words that convey meaning
- ➋ Common words with no informational value (e.g., "the", "a", "also")
- ➌ All nouns and verbs
- ➍ All proper nouns

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Stop words definition:**
  - Words that have little informational value
  - Included to make sentences flow
  - Examples: "has", "a", "the", "also", "and", "of"
  - Common in all documents
- **Why remove them:**
  - They add noise to analysis
  - They don't help distinguish documents
  - They increase vector size unnecessarily
  - Improve computational efficiency
- **Caveat:**
  - What constitutes a stop word varies by application
  - Words with no value in one context may be useful in another
  - Customization may be needed

**EXAM TRAP:** Stop words = common words with no informational value!

# Q13: Stemming Definition - Tutorial

**QUESTION:** What is stemming in NLP pre-processing?

**OPTIONS:**

- ❶ Removing all words from a document
- ❷ Replacing words with their core/root forms by cutting prefixes and suffixes
- ❸ Converting all text to uppercase
- ❹ Removing all punctuation

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Stemming definition:**
  - Reduces words to their root forms
  - Cuts off prefixes and suffixes
  - Crude, rule-based approach
- **Examples:**
  - "disappointing" → "disappoint"
  - "disappointed" → "disappoint"
  - "running" → "run"
  - "jumping" → "jump"
- **Purpose:**
  - Treat similar words equally in analysis
  - Reduce vocabulary size
  - Simplify analysis
- **Limitation:**
  - Can lose nuance and degrees of positivity/negativity
  - May create non-words
  - Less accurate than lemmatization

**EXAM TRAP:** Stemming = cutting prefixes/suffixes to root form!

# Q14: Lemmatization Definition - Tutorial

**QUESTION:** What is lemmatization and how does it differ from stemming?

**OPTIONS:**

- ① Lemmatization is the same as stemming
- ② Lemmatization finds the dictionary form (lemma) considering word meaning and part of speech
- ③ Lemmatization removes stop words
- ④ Lemmatization converts text to lowercase

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Lemmatization definition:**
  - Finds the dictionary form (lemma) of words
  - Considers word meaning and part of speech
  - More sophisticated than stemming
- **Comparison with stemming:**
  - **Stemming:**
    - Rule-based, cuts prefixes/suffixes
    - Fast, but can be inaccurate
    - May create non-words
  - **Lemmatization:**
    - Uses vocabulary and morphological analysis
    - Slower but more accurate
    - Always produces real words
- **Examples:**
  - "running" → "run" (lemma)
  - "better" → "good" (lemma)
  - "went" → "go" (lemma)

**EXAM TRAP:** Lemmatization = dictionary form considering meaning and POS!

# Q15: POS Tagging Purpose - Tutorial

**QUESTION:** What is Part-of-Speech (POS) tagging and why is it useful?

**OPTIONS:**

- ❶ Identifying all verbs in a document
- ❷ Labelling each word by its class (noun, verb, adjective, proper noun, etc.)
- ❸ Removing all adjectives from a document
- ❹ Counting the number of sentences

**ANSWER: B**

**DETAILED TUTORIAL:**

- **POS tagging definition:**
  - Labels each word with its grammatical category
  - Common tags: noun, verb, adjective, proper noun, adverb
  - Provides syntactic information
- **Why it's useful:**
  - **Handles ambiguity:**
    - "Reading" (town) = proper noun
    - "reading" (activity) = verb
  - **Handles case sensitivity:**
    - "Polish" (language) = proper noun
    - "polish" (to shine) = verb
  - **Improves accuracy:**
    - Identifies proper nouns before lowercasing
    - Preserves important distinctions

**EXAM TRAP:** POS tagging labels **each word by grammatical class!**



**QUESTION:** What problem can lowercasing cause in NLP?

**OPTIONS:**

- ❶ It makes text harder to read
- ❷ It can cause inaccuracies by removing case-based distinctions (proper nouns, different meanings)
- ❸ It doesn't affect NLP
- ❹ It makes words longer

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Lowercasing problems:**

❶ **Proper nouns:**

- "Reading" (town) vs "reading" (activity)
- "Polish" (language) vs "polish" (to shine)
- Names of people, places, firms

❷ **Meaning loss:**

- Important distinctions are lost
- Can lead to misclassification
- Reduces accuracy

• **Solution:**

- Apply POS tagging before lowercasing
- Preserve proper noun information
- Maintain case-based distinctions

**EXAM TRAP:** Lowercasing can lose **proper noun distinctions** - use POS tagging first!

# Q17: Punctuation Removal Effects - Tutorial

**QUESTION: What is a limitation of removing punctuation?**

**OPTIONS:**

- ❶ It doesn't affect NLP
- ❷ It can cause loss of valuable context (questions, exclamations, sarcasm)
- ❸ It makes text shorter
- ❹ It removes all verbs

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Punctuation problems:**

❶ **Questions:**

- "Is the CEO performing well?" → may seem positive
- Without "?", classified as positive
- Actually a question (little information content)

❷ **Exclamations:**

- May denote humor or sarcasm
- Different information value than period
- Tone indication is lost

• **Solution:**

- Consider punctuation separately
- Identify question marks, exclamation points
- Use as additional features
- Context matters

**EXAM TRAP:** Removing punctuation loses **context** and **tone** information!

# Q18: Stemming vs Lemmatization Usage - Tutorial

**QUESTION:** When might lemmatization be preferred over stemming?

**OPTIONS:**

- ❶ When speed is the only concern
- ❷ When accuracy and preserving meaning are important
- ❸ When working with very short documents
- ❹ When speed is critical

**ANSWER: B**

**DETAILED TUTORIAL:**

- **When to use lemmatization:**
  - **Accuracy is critical:**
    - Financial documents need precise analysis
    - Proper word forms matter
    - Avoid non-words
  - **Meaning preservation:**
    - Different forms of same word
    - Irregular verbs
    - Context-specific forms
- **When to use stemming:**
  - **Speed is critical:**
    - Very large documents
    - Real-time processing
    - Simpler analysis
- **Tradeoff:**
  - Stemming: faster, less accurate
  - Lemmatization: slower, more accurate

**EXAM TRAP:** Lemmatization preferred for **accuracy**, stemming for **speed**!

# Q19: Vocabulary Building - Tutorial

**QUESTION:** How is the vocabulary for BoW determined?

**OPTIONS:**

- ❶ Only using pre-existing dictionaries
- ❷ Either imposed exogenously or created from every distinct word in the corpus
- ❸ Only using the most common words
- ❹ Only using the rarest words

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Vocabulary creation methods:**

❶ **Exogenous creation:**

- Created ahead of time
- Applied to documents
- Fixed and consistent

❷ **Corpus-based creation:**

- Every distinct word in all documents
- Comprehensive and complete
- May be very large

• **Tradeoffs:**

- **Exogenous:** Controlled but may miss domain-specific words
- **Corpus-based:** Complete but may be too large

• **Processing:**

- Remove very frequent words (top 5%)
- Remove very rare words
- Balance vocabulary size

**EXAM TRAP:** Vocabulary = [pre-defined](#) OR [from corpus words](#)!

**QUESTION: What is the Bag of Words (BoW) approach?**

**OPTIONS:**

- ❶ Analyzing words in their original order
- ❷ Treating each word in a document independently, ignoring order
- ❸ Counting only the longest words
- ❹ Removing all words from documents

**ANSWER: B**

**DETAILED TUTORIAL:**

- **BoW definition:**
  - Treats each word distinctly and equally
  - Position and order are ignored
  - Counts occurrences in each document
- **Key characteristics:**
  - **Independence:** Each word considered separately
  - **Equal treatment:** All words have same importance (in basic version)
  - **Simplification:** Removes complex language structure
- **Limitations:**
  - Loses context
  - Ignores negation words
  - Misses phrase meanings
  - n-grams can help address this

**EXAM TRAP:** BoW ignores word order and position!

# Q21: Binary vs Count BoW - Tutorial

**QUESTION:** What is the difference between binary and count Bag of Words?

**OPTIONS:**

- ① Binary uses 1/0, count uses frequency counts
- ② Binary is more accurate than count
- ③ Count ignores word frequencies
- ④ There is no difference

**ANSWER: A**

**DETAILED TUTORIAL:**

- **Binary BoW:**
  - 1 if word appears in document
  - 0 if word does not appear
  - Presence/absence only
  - **Example:** [1, 0, 1, 0, ...]
- **Count BoW:**
  - Counts how many times each word appears
  - Non-negative integers
  - Frequency matters
  - **Example:** [2, 0, 1, 0, ...]
- **When to use each:**
  - **Binary:** When presence is enough
  - **Count:** When frequency matters
  - Usually count BoW gives more information

**EXAM TRAP:** Binary = **presence/absence**, Count = **frequency counts**!

**QUESTION:** What is a Document Term Matrix (DTM)?

**OPTIONS:**

- ❶ A matrix of document titles
- ❷ A collection of document vectors with rows as documents and columns as terms
- ❸ A matrix of document dates
- ❹ A matrix of document authors

**ANSWER:** B

**DETAILED TUTORIAL:**

- **DTM definition:**
  - Rows = documents
  - Columns = terms/words
  - Entries = word counts or frequencies
  - Dimensions:  $|W| \times D$
- **Characteristics:**
  - **Sparse:** Many zeros
  - **Large:** Many columns (vocabulary size)
  - **Quantitative:** Enables analysis
- **Example:**
  - 1,000 documents, 10,000 word vocabulary
  - Matrix:  $1,000 \times 10,000$
  - Most entries are zero

**EXAM TRAP:** DTM = documents as rows, terms as columns!

## Q23: Sparse Vectors Problem - Tutorial

**QUESTION:** Why are document word vectors described as sparse?

**OPTIONS:**

- ① They contain many non-zero values
- ② They contain many zeros
- ③ They are all the same length
- ④ They are very small

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Sparsity definition:**
  - Most words don't appear in each document
  - Vector entries are mostly zero
  - Common in NLP due to large vocabularies
- **Example:**
  - Vocabulary: 10,000 words
  - Document: 100 unique words
  - 9,900 zeros in vector
  - 99% sparsity
- **Problems:**
  - **Computational inefficiency:**
    - Slow processing
    - Memory intensive
  - **Statistical issues:**
    - Curse of dimensionality
    - Hard to find patterns
- **Solutions:**
  - Dimensionality reduction
  - Feature selection
  - Sparse matrix representations

**EXAM TRAP:** Sparse = **many zeros** - makes modeling inefficient!



## Q24: Vector Normalization - Tutorial

**QUESTION:** Why is vector normalization needed in NLP?

**OPTIONS:**

- ❶ To make vectors longer
- ❷ To prevent repeated words from skewing analysis
- ❸ To remove all words
- ❹ To make vectors shorter

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Why normalize:**

- Repeated words can skew analysis
- Example: "Bad" repeated multiple times
- Similar to outliers in econometrics
- Need to equalize influence

• **L2 normalization:**

- Divide each element by the square root of sum of squares
- $\|v\|_2 = \sqrt{\sum x_i^2}$
- Scales elements between 0 and 1
- All documents become unit length

• **Example:**

- Original: [2, 0, 1, 0, 1]
- L2 norm:  $2^2 + 0 + 1 + 0 + 1 = 6$
- Normalized:  $[2/\sqrt{6}, 0, 1/\sqrt{6}, 0, 1/\sqrt{6}]$
- $\approx [0.82, 0, 0.41, 0, 0.41]$

**EXAM TRAP:** L2 normalization = [scale to unit length!](#)

## Q25: L2 Norm Formula - Tutorial

**QUESTION:** What is the formula for the L2 norm of a vector?

**OPTIONS:**

- ❶  $||X||_2 = \sum x_i$
- ❷  $||X||_2 = \sqrt{\sum x_i^2}$
- ❸  $||X||_2 = \sum x_i^2$
- ❹  $||X||_2 = \max(x_i)$

**ANSWER: B**

**DETAILED TUTORIAL:**

• **L2 norm formula:**

- $||X||_2 = \sqrt{\sum_{i=1}^n x_i^2}$
- Where  $x_i$  are elements of the vector
- Also called Euclidean norm

• **Application in NLP:**

- Normalize document vectors
- Each vector becomes unit length
- Comparable across documents

• **Example calculation:**

- Vector:  $[2, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1]$
- Sum of squares  $= 4 + 1 + 1 + 1 + 1 = 8$
- L2 norm  $= \sqrt{8} = 2.83$

**EXAM TRAP:** L2 norm = square root of sum of squared elements!

**QUESTION:** What is the dictionary approach in NLP?

**OPTIONS:**

- ① A machine learning algorithm
- ② A pre-defined list of words sharing a common characteristic used for text analysis
- ③ A method for removing stop words
- ④ A way to tokenize documents

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Dictionary approach definition:**
  - Pre-defined list of words
  - Words share a common characteristic
  - Used to assess sentiment or topic
- **Common application:**
  - Sentiment analysis
  - Separate positive and negative words
  - Count occurrences in documents
- **Net sentiment score:**
  - Proportion positive - proportion negative
  - Divide each by total words
  - Neutral words can be ignored

**EXAM TRAP:** Dictionary = pre-defined word lists with common characteristics!

# Q27: Dictionary Approach Example - Tutorial

**QUESTION:** In the newswire example, how is sentiment determined?

**OPTIONS:**

- ① Count only positive words
- ② Count only negative words
- ③ Count positive and negative words, subtract negatives from positives
- ④ Count all words equally

**ANSWER: C**

**DETAILED TUTORIAL:**

• **Sentiment calculation:**

- Count positive words: rise, grow, resolve, relief (4 positives)
- Count negative words: disappoint, worry, concern, decline (4 negatives)
- Net sentiment =  $4 - 4 = 0$  (neutral)

• **Example text:**

- "Firm XYZ reported rise in earnings, disappointing investors"
- "Previous safety worries resolved, should underpin growth"
- "Analyst expressed relief, concerns about decline"

• **Challenges:**

- Counterfactual sentences ("would have led to decline")
- Complex sentence structure
- Negation words can reverse meaning

**EXAM TRAP:** Net sentiment = positive count - negative count!

## Q28: Loughran-McDonald Dictionary - Tutorial

**QUESTION:** What is the Loughran-McDonald dictionary known for?

**OPTIONS:**

- ① Being the largest dictionary
- ② Being specifically designed for financial text analysis
- ③ Being the oldest dictionary
- ④ Only containing positive words

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Loughran-McDonald dictionary:**
  - Developed for financial text analysis
  - Based on extensive examination of 10-K filings
  - Categories: positive, negative, uncertain, litigious, strong modal, weak modal
- **Accuracy:**
  - Around 75% accuracy on Thomson Reuters news articles
  - Widely used in finance research
- **Importance:**
  - General dictionaries misclassify financial text
  - Financial words have specific meanings
  - Example: "liability" is not negative in financial context

**EXAM TRAP:** Loughran-McDonald = financial text dictionary - 75% accuracy!

**QUESTION:** What are the primary advantages of the dictionary approach?

**OPTIONS:**

- ❶ It's always 100% accurate
- ❷ Transparency, ease and speed of implementation
- ❸ It works for all types of text
- ❹ It requires no pre-processing

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Key advantages:**

❶ **Transparency:**

- Word lists are explicit and clear
- Easy to understand what's being measured
- Results are interpretable

❷ **Ease of implementation:**

- Simple word counting
- No training required
- Fast to apply

❸ **Comparability:**

- Same word lists across corpora
- Consistent measurement
- Enables comparison

**EXAM TRAP:** Advantages = transparency, ease, speed, comparability!

# Q30: Dictionary Approach Limitations - Tutorial

**QUESTION: What is a key limitation of the dictionary approach?**

**OPTIONS:**

- ❶ It's too fast
- ❷ Dictionaries are only as good as the word lists they are built on
- ❸ It always overestimates sentiment
- ❹ It always underestimates sentiment

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Key limitations:**

❶ **Quality dependence:**

- Only as good as word lists
- Incomplete lists → poor accuracy
- Ambiguities → misclassification
- Errors propagate

❷ **Context limitation:**

- Designed for formal documents
- Poor for social media
- Colloquial language not captured
- "Moon" not in financial dictionaries

❸ **Narrow scope:**

- Limited subjects available
- Time-consuming to create new dictionaries
- Machine learning often preferred for new applications

❹ **Negation issues:**

- "Not good" → should be negative
- "Not bad" → should be positive
- Negations away from objects are hard

**EXAM TRAP:** Dictionary = **only as good as word lists** - has many limitations!

# Q31: n-gram Definition - Tutorial

**QUESTION:** What is an n-gram in NLP?

**OPTIONS:**

- ❶ A single word
- ❷ A sequence of  $n$  contiguous words treated as a single unit
- ❸ A type of machine learning algorithm
- ❹ A pre-processing technique

**ANSWER: B**

**DETAILED TUTORIAL:**

• **n-gram definition:**

- $n = 1$ : uni-gram (single word)
- $n = 2$ : bi-gram (two words)
- $n = 3$ : tri-gram (three words)
- Treats contiguous words as single unit

• **Example:**

- Document: "credit spreads narrowed today"
- Uni-grams: ["credit", "spreads", "narrowed", "today"]
- Bi-grams: ["credit spreads", "spreads narrowed", "narrowed today"]

• **Purpose:**

- Capture phrase meanings
- Handle negation words
- Address context issues

**EXAM TRAP:** n-gram =  $n$  contiguous words as one unit!



## Q32: Why Use n-grams? - Tutorial

**QUESTION: Why are n-grams needed beyond basic BoW?**

**OPTIONS:**

- ❶ To make analysis faster
- ❷ Because some words have specific meaning when placed together
- ❸ Because basic BoW is too accurate
- ❹ To count words more precisely

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Limitations of basic BoW:**

❶ **Loses phrase meaning:**

- "red herring"  $\neq$  red + herring
- "San Diego"  $\neq$  San + Diego
- "credit spread"  $\neq$  credit + spread

❷ **Negation words:**

- "not good"  $\neq$  not + good
- "nothing useful"  $\neq$  nothing + useful
- "neither up nor down"  $\neq$  neither + up + nor + down

• **How n-grams help:**

- Preserve phrase meaning
- Handle negations correctly
- Capture context
- More accurate sentiment analysis

**EXAM TRAP:** n-grams capture **phrase meaning and negation context!**

# Q33: Negation Handling with n-grams - Tutorial

**QUESTION:** How can n-grams help with negation words?

**OPTIONS:**

- ❶ By removing all negation words
- ❷ By treating "not" + following word as a single unit
- ❸ By ignoring negations
- ❹ By making negations positive

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Negation problem:**

- "not good" → should be negative sentiment
- Basic BoW sees "not" and "good" separately
- Misses the negation effect

• **n-gram solution:**

- Bi-grams capture "not good" as one unit
- "not sufficient" as one unit
- "never great" as one unit
- Preserves negation meaning

• **Example:**

- "The service is not good"
- Bi-grams: ["The service", "service is", "is not", "not good"]
- "not good" detected as negative phrase

**EXAM TRAP:** n-grams pair **negation** + **following word** as one unit!

**QUESTION:** What does TF-IDF stand for and what does it measure?

**OPTIONS:**

- ① Term Frequency - Inverse Document Frequency; weights words by importance
- ② Total Frequency - Integrated Document Frequency; measures total occurrences
- ③ Text Frequency - Inverse Data Frequency; measures text length
- ④ Term Frequency - Inverse Data Frequency; measures document similarity

**ANSWER: A**

**DETAILED TUTORIAL:**

- **TF-IDF definition:**
  - **TF (Term Frequency):** How often word appears in document
  - **IDF (Inverse Document Frequency):** How rare the word is across documents
  - **TF-IDF:**  $TF \times IDF = \text{word importance score}$
- **Purpose:**
  - Assign higher weights to important words
  - Words frequent in document but rare in corpus = high score
  - Words frequent everywhere = low score
- **Intuition:**
  - Common words ("the", "this") get zero weight
  - Rare but relevant words get high weight
  - Better than simple word counts

**EXAM TRAP:**  $TF-IDF = \text{Term Frequency} \times \text{Inverse Document Frequency!}$

# Q35: Term Frequency Formula - Tutorial

**QUESTION:** What is the formula for Term Frequency (TF)?

**OPTIONS:**

- ❶  $TF_{i,j} = w_{i,j} \times |v_j|$
- ❷  $TF_{i,j} = w_{i,j} / |v_j|$
- ❸  $TF_{i,j} = |v_j| / w_{i,j}$
- ❹  $TF_{i,j} = \log(w_{i,j})$

**ANSWER: B**

**DETAILED TUTORIAL:**

• **TF Formula:**

- $TF_{i,j} = \frac{w_{i,j}}{|v_j|}$
- $w_{i,j}$ : number of times term  $i$  appears in document  $j$
- $|v_j|$ : total number of terms in document  $j$
- Normalizes by document length

• **Why normalize:**

- Longer documents have more words
- Without normalization, longer documents dominate
- Corrects for document length

• **Example:**

- Word appears 3 times in 100-word document
- $TF = 3/100 = 0.03$
- Word appears 3 times in 50-word document
- $TF = 3/50 = 0.06$  (higher importance)

**EXAM TRAP:**  $TF = \text{word count} / \text{total words in document!}$

## Q36: Inverse Document Frequency Formula - Tutorial

**QUESTION:** What is the formula for Inverse Document Frequency (IDF)?

**OPTIONS:**

- ①  $IDF_i = \ln(D / \sum d_{i,j})$
- ②  $IDF_i = D / \sum d_{i,j}$
- ③  $IDF_i = \sum d_{i,j} / D$
- ④  $IDF_i = \ln(\sum d_{i,j} / D)$

**ANSWER: A**

**DETAILED TUTORIAL:**

• **IDF Formula:**

- $IDF_i = \ln(D / \sum_j d_{i,j})$
- $D$  = total number of documents
- $\sum d_{i,j}$  = number of documents containing term  $i$
- Uses natural logarithm

• **Interpretation:**

- Rare words: High IDF
- Common words: Low IDF
- Words in all documents:  $IDF = 0$

• **Example:**

- 100 documents, word appears in 10 documents
- $IDF = \ln(100/10) = \ln(10) = 2.30$
- Word appears in 90 documents
- $IDF = \ln(100/90) = \ln(1.11) = 0.10$

**EXAM TRAP:**  $IDF = \ln(\text{total documents} / \text{documents containing word})!$

# Q37: Zero TF-IDF Value - Tutorial

**QUESTION: Why do some words have zero TF-IDF values?**

**OPTIONS:**

- ❶ Because they are not in the vocabulary
- ❷ Because they appear in every document (no discriminatory power)
- ❸ Because they are too long
- ❹ Because they are too short

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Zero IDF explanation:**

- $IDF = \ln(D / \sum d_{i,j})$
- If word appears in all documents:  $\sum d_{i,j} = D$
- $IDF = \ln(D/D) = \ln(1) = 0$
- $TF-IDF = TF \times 0 = 0$

• **Why this matters:**

- Word has no discriminatory power
- Cannot distinguish between documents
- No value for classification

• **Example:**

- "this" appears in all 4 documents in Box 9.2
- $IDF = \ln(4/4) = 0$
- $TF-IDF = 0$

**EXAM TRAP:** Zero TF-IDF = word appears in all documents - no discriminatory power!

## Q38: TF-IDF Variation by Document - Tutorial

**QUESTION: Why does the same word have different TF-IDF values across documents?**

**OPTIONS:**

- ❶ IDF varies by document
- ❷ TF varies by document (due to different lengths and frequencies)
- ❸ TF-IDF is always constant
- ❹ Vocabulary changes

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Why TF varies:**

❶ **Document length:**

- TF depends on document length
- Longer documents have smaller TFs
- Same word count, different TF

❷ **Frequency:**

- Word may appear multiple times in one document
- Once in another document
- TF changes accordingly

• **IDF is constant:**

- IDF depends only on corpus
- Same for all documents
- TF varies by document
- $\text{TF-IDF} = \text{TF} \times \text{IDF} \Rightarrow \text{varies by document}$

**EXAM TRAP:** IDF is constant across documents; TF varies  $\Rightarrow$  TF-IDF varies!

**QUESTION:** What are common variations in TF-IDF implementation?

**OPTIONS:**

- ❶ Only one formula exists
- ❷ Variations include binary TF, log normalization, and different IDF formulas
- ❸ TF-IDF is always the same
- ❹ TF-IDF cannot be modified

**ANSWER: B**

**DETAILED TUTORIAL:**

❶ **Common variations:**

❶ **TF variations:**

- Binary TF: 1 if word appears, 0 otherwise
- Raw count:  $w_{i,j}$  (no normalization)
- Log normalization:  $\ln(1 + w_{i,j})$
- Normalized:  $w_{i,j} / |v_j|$  (standard)

❷ **IDF variations:**

- Smooth IDF:  $\ln((D+1)/(\sum d_{i,j}+1)) + 1$
- Standard IDF:  $\ln(D/\sum d_{i,j})$
- Probabilistic IDF:  $\log((D - \sum d_{i,j})/\sum d_{i,j})$

❸ **Why variations exist:**

- Different applications need different emphasis
- Some handle document length better
- Some are more robust
- Different software implementations

**EXAM TRAP:** TF-IDF has **many implementation variations** - be aware!



## Q40: TF-IDF Example Interpretation - Tutorial

**QUESTION:** In the TF-IDF example, which word has the highest score and why?

**OPTIONS:**

- ❶ "this" - because it appears in all documents
- ❷ "fed" - because it appears in only one short document
- ❸ "down" - because it appears frequently
- ❹ "stock" - because it appears in two documents

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Highest scores:**

- "fed" = 0.28 (TF-IDF)
- "hikes" = 0.28 (TF-IDF)
- "rates" = 0.28 (TF-IDF)

• **Why they're highest:**

- Appear in only 1 out of 4 documents
- $IDF = \ln(4/1) = 1.39$  (high)
- Document length = 5 (short)
- $TF = 1/5 = 0.20$
- $TF-IDF = 0.20 \times 1.39 = 0.28$

• **Lowest scores:**

- "this" = 0 (appears in all documents)
- "down" = 0.05 (appears in 3 of 4 documents)

**EXAM TRAP:** Rare words in short documents = **highest TF-IDF!**

# Q41: Naïve Bayes Definition - Tutorial

**QUESTION:** What is the Naïve Bayes classifier?

**OPTIONS:**

- ① A regression algorithm
- ② A classification algorithm based on Bayes' theorem with independence assumption
- ③ A clustering algorithm
- ④ A dimensionality reduction technique

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Naïve Bayes definition:**
  - Based on Bayes' theorem
  - Assumes features are independent (naïve assumption)
  - Used for classification tasks
  - Popular for text classification
- **Why "naïve":**
  - Assumes word independence
  - In reality, words are not independent
  - Still works well in practice
- **Advantages:**
  - Simple and fast
  - Solid classification accuracy
  - Works well with high-dimensional data
  - Easy to implement

**EXAM TRAP:** Naïve = **independence assumption** - words treated independently!

**QUESTION:** What is Bayes' theorem in the context of classification?

**OPTIONS:**

- ①  $P(c|d) = P(d|c)P(c)/P(d)$
- ②  $P(c|d) = P(d|c) + P(c)$
- ③  $P(c|d) = P(c)/P(d)$
- ④  $P(c|d) = P(d)/P(c)$

**ANSWER: A**

**DETAILED TUTORIAL:**

● **Bayes' Theorem:**

- $P(c|d) = \frac{P(d|c) \times P(c)}{P(d)}$
- $P(c|d)$ : Posterior probability (probability of class given document)
- $P(d|c)$ : Likelihood (probability of document given class)
- $P(c)$ : Prior probability (unconditional probability of class)
- $P(d)$ : Predictor prior probability (normalizing constant)

● **Application:**

- Calculate probability for each class
- Assign document to class with highest probability
- $P(d)$  cancels across classes (can be ignored)

**EXAM TRAP:** Bayes = posterior = likelihood  $\times$  prior / evidence!

# Q43: Naïve Independence Assumption - Tutorial

**QUESTION:** What is the independence assumption in Naïve Bayes?

**OPTIONS:**

- ① Documents are independent of each other
- ② Each word in a document is independent of all other words
- ③ Classes are independent of documents
- ④ All documents have the same length

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Independence assumption:**
  - $P(w_1, w_2, \dots, w_n|c) = P(w_1|c) \times P(w_2|c) \times \dots \times P(w_n|c)$
  - Words are assumed conditionally independent
  - Given the class, word probabilities multiply
- **Why it's naïve:**
  - In reality, words are not independent
  - "good" and "great" often appear together
  - "not" and "good" are related
- **Why it works:**
  - Despite unrealistic assumption
  - Works well in practice
  - Simple and effective

**EXAM TRAP:** Independence = words treated as independent - unrealistic but works!

## Q44: Prior Probability - Tutorial

**QUESTION:** How is prior probability  $P(c)$  calculated in Naïve Bayes?

**OPTIONS:**

- ①  $P(c) = n(c)/N$  (documents in class / total documents)
- ②  $P(c) = N/n(c)$
- ③  $P(c) = n(c) \times N$
- ④  $P(c) = 1/N$

**ANSWER: A**

**DETAILED TUTORIAL:**

• **Prior probability formula:**

- $P(c_i) = \frac{n(c_i)}{N}$
- $n(c_i)$ : number of documents labeled as class  $i$
- $N$ : total number of documents
- Unconditional probability of each class

• **Example:**

- 8 total documents
- 4 good, 2 bad, 2 indifferent
- $P(\text{good}) = 4/8 = 0.5$
- $P(\text{bad}) = 2/8 = 0.25$
- $P(\text{indifferent}) = 2/8 = 0.25$

• **Importance:**

- Prior knowledge about class frequencies
- Updates to posterior probability
- Helps when likelihoods are similar

**EXAM TRAP:** Prior = class count / total documents!

**QUESTION:** How is the likelihood  $P(w|c)$  calculated in Naïve Bayes?

**OPTIONS:**

- ① Count of word occurrences in class / total words in class
- ② Count of documents in class / total documents
- ③ Word count / total words in corpus
- ④ Always 0.5

**ANSWER: A**

**DETAILED TUTORIAL:**

• **Likelihood calculation:**

- $P(w_i|c_j) = \frac{\text{count of } w_i \text{ in class } c_j}{\text{total words in class } c_j}$
- Conditional probability of word given class
- Estimated from labeled training data

• **Example from customer service:**

- Class 1 (good): 4 documents
- Word 2 appears in 1 of 4 good documents
- $P(w_2|good) = 1/4 = 0.25$

• **Important:**

- Zero probabilities are problematic
- Need smoothing to avoid zeros

**EXAM TRAP:** Likelihood = word count in class / total words in class!

# Q46: Posterior Probability - Tutorial

**QUESTION:** What is the posterior probability in Naïve Bayes?

**OPTIONS:**

- ❶ The probability of a document given its class
- ❷ The probability of a class given the document
- ❸ The probability of a word given its class
- ❹ The unconditional class probability

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Posterior definition:**
  - $P(c|d)$
  - Probability that document  $d$  belongs to class  $c$
  - Updated after seeing the document
  - Combines prior and likelihood
- **Calculation:**
  - $P(c|d) \propto P(d|c) \times P(c)$
  - Compute for each class
  - Choose class with highest posterior
- **Example:**
  - After seeing words 1 and 2
  - $P(bad|words) = 0.67$
  - $P(indifferent|words) = 0.33$
  - $P(good|words) = 0$
  - Classify as "bad"

**EXAM TRAP:** Posterior = probability of class given document!

**QUESTION:** What is Maximum A Posteriori (MAP) classification?

**OPTIONS:**

- ① Assign document to the class with the highest posterior probability
- ② Assign document to the class with the highest prior probability
- ③ Assign document to the class with the highest likelihood
- ④ Random classification

**ANSWER: A**

**DETAILED TUTORIAL:**

• **MAP definition:**

- $\hat{c} = \arg \max_{c \in C} P(c|d)$
- Choose class that maximizes posterior
- "Maximum A Posteriori"

• **Simplified form:**

- $\hat{c} = \arg \max_{c \in C} P(d|c) \times P(c)$
- $P(d)$  cancels out
- Only need likelihood and prior

• **Example:**

- $P(bad|words) = 0.67$
- $P(indifferent|words) = 0.33$
- $P(good|words) = 0$
- MAP class = "bad"

**EXAM TRAP:** MAP = class with highest posterior probability!



# Q48: Zero Probability Problem - Tutorial

**QUESTION:** What is the zero probability problem in Naïve Bayes?

**OPTIONS:**

- ① When a word appears in all documents
- ② When a word does not appear in a class in the training data, making the class probability zero
- ③ When all words have zero frequency
- ④ When no documents exist

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Zero probability problem:**
  - Word doesn't appear in training data for a class
  - $P(w|c) = 0$
  - Product becomes zero:  $\prod P(w_i|c) = 0$
  - Class probability becomes zero
- **Example:**
  - No "good" reviews contain word "slow"
  - New document has "slow" and otherwise positive words
  - Probability of "good" = 0
  - But review could still be good!
- **Consequences:**
  - Overly draconian
  - Unrealistic zero probabilities
  - Can misclassify documents

**EXAM TRAP:** Zero probability = word missing in training class - need smoothing!

**QUESTION: What is Laplace smoothing in Naïve Bayes?**

**OPTIONS:**

- ❶ A method to remove stop words
- ❷ Adding 1 to all counts to avoid zero probabilities
- ❸ A technique to stem words
- ❹ A method to normalize vectors

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Laplace smoothing:**

- Adds 1 (or  $\lambda$ ) to all counts
- Ensures all probabilities are non-zero
- $P(w|c) = \frac{\text{count}(w,c)+1}{\text{total words in } c+|V|}$

• **Effect:**

- Zero probabilities become positive
- All probabilities pushed toward uniform
- Larger  $\lambda$  = stronger smoothing
- Impact decreases with sample size

• **Example:**

- Before:  $P(w|c) = 0/4 = 0$
- After:  $P(w|c) = 1/6 = 0.167$
- Probability increases from 0 to small positive value

**EXAM TRAP:** Laplace smoothing = add 1 to all counts to avoid zeros!

## Q50: Smoothing Impact on Probabilities - Tutorial

**QUESTION:** How does smoothing affect the estimated probabilities?

**OPTIONS:**

- ① It makes probabilities more extreme
- ② It pushes probabilities toward a uniform distribution
- ③ It has no effect on probabilities
- ④ It makes probabilities negative

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Effect of smoothing:**
  - Prevents zero probabilities
  - Shrinks probabilities toward uniform
  - Reduces extreme values
  - Similar to regularization (Lasso/Ridge)
- **Impact of  $\lambda$ :**
  - $\lambda = 0$ : No smoothing (original)
  - $\lambda = 1$ : Laplace smoothing (common)
  - $\lambda > 1$ : Stronger smoothing
  - Larger  $\lambda$  = more uniform probabilities
- **Sample size effect:**
  - Small sample: Smoothing has large impact
  - Large sample: Smoothing has small impact
  - Effect diminishes with more data

**EXAM TRAP:** Smoothing = **shrinks toward uniform distribution** - similar to regularization!

# Q51: Underflow Problem - Tutorial

**QUESTION:** What is the underflow problem in Naïve Bayes?

**OPTIONS:**

- ① When probabilities become too large
- ② When multiplying many small probabilities results in numbers too small for computers to handle
- ③ When documents are too long
- ④ When vocabulary is too small

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Underflow definition:**
  - Multiplying many probabilities  $< 1$
  - Product becomes extremely small
  - Below computer precision
  - Rounds to zero
- **Cause:**
  - $P(w_1|c) \times P(w_2|c) \times \dots \times P(w_n|c)$
  - Each term  $< 1$
  - For long documents, product  $\rightarrow 0$
  - Example:  $0.5^{100} \approx 7.9 \times 10^{-31}$
- **Solution:**
  - Take logarithms
  - $\log(P(w_1|c) \times \dots) = \sum \log(P(w_i|c))$
  - Sums instead of products
  - Computationally stable

**EXAM TRAP:** Underflow = [multiplying small probabilities](#) - use logarithms!

## Q52: Log Transformation - Tutorial

**QUESTION:** Why is log transformation used in Naïve Bayes?

**OPTIONS:**

- ➊ To make probabilities larger
- ➋ To convert products to sums and avoid underflow
- ➌ To remove stop words
- ➍ To normalize the data

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Log transformation benefits:**

➊ **Products to sums:**

- $\log(P_1 \times P_2 \times \dots) = \log(P_1) + \log(P_2) + \dots$
- Avoids multiplication of small numbers
- Computationally stable

➋ **Preserves ordering:**

- Log is monotonic
- Class with highest probability = class with highest log probability
- Results unchanged

➌ **Simplifies calculations:**

- Sums are easier to compute
- Less risk of numerical issues
- More efficient

• **Example:**

- Instead of:  $0.5 \times 0.25 \times 0.33$
- Use:  $\log(0.5) + \log(0.25) + \log(0.33)$

**EXAM TRAP:** Log = **products to sums** - avoids underflow!

# Q53: Unknown Words Handling - Tutorial

**QUESTION:** How are words in test data that were not in training data handled?

**OPTIONS:**

- ① They are added to the vocabulary
- ② They are ignored because they cannot help classify the document
- ③ They cause the classification to fail
- ④ They are treated as stop words

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Unknown word handling:**
  - Words not in training vocabulary
  - No information about their distribution
  - Cannot help with classification
  - Must be removed from test document
- **Why ignore them:**
  - $P(w|c)$  unknown
  - Cannot calculate likelihood
  - Would cause problems
  - Better to exclude them
- **Solution:**
  - Build vocabulary from training data
  - Only use known words for classification
  - Unknown words provide no discriminatory power

**EXAM TRAP:** Unknown words = **ignored** - they provide no classification information!

**QUESTION:** What are the two key assumptions of Naïve Bayes for document classification?

**OPTIONS:**

- ① Words are dependent and have different weights
- ② Words are independent and have equal weighting
- ③ Words are ordered and have different weights
- ④ Words are random and have no weights

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Assumption 1: Independence**
  - Each word is independent of others
  - $P(w_1, w_2, \dots, w_n|c) = \prod P(w_i|c)$
  - Simplifies calculations
- **Assumption 2: Equal weighting**
  - Each word has equal information value
  - No word is more important than others
  - After stop word removal
- **Why these assumptions:**
  - Make problem tractable
  - Work surprisingly well
  - Simple and fast

**EXAM TRAP:** Naïve Bayes = independence + equal weighting!

# Q55: Generative vs Discriminative - Tutorial

**QUESTION:** What makes Naïve Bayes a generative classifier?

**OPTIONS:**

- ❶ It can generate new documents
- ❷ It learns the joint distribution  $P(d, c)$  and can create new classifications
- ❸ It only learns decision boundaries
- ❹ It doesn't use probabilities

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Generative classifiers:**
  - Model the joint distribution  $P(d, c)$
  - Learn how data is generated
  - Can generate new examples
  - Examples: Naïve Bayes, Hidden Markov Models
- **Discriminative classifiers:**
  - Model the decision boundary  $P(c|d)$  directly
  - Only care about classification
  - Examples: Logistic regression, SVM
- **Difference:**
  - Generative: Complete model of data
  - Discriminative: Only classification boundary
  - Generative can create new classifications

**EXAM TRAP:** Naïve Bayes = **generative** - learns joint distribution!



**QUESTION:** In the customer service example, what classification is made and why?

**OPTIONS:**

- ① "Good" because it has the highest prior
- ② "Bad" because it has the highest posterior probability
- ③ "Indifferent" because it's the middle class
- ④ Cannot be classified

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Data:**
  - 4 good, 2 bad, 2 indifferent (8 total)
  - New document: words 1 and 2 present, words 3 and 4 absent
- **Calculated probabilities:**
  - $P(\text{good}|\text{words}) = 0$  (without smoothing)
  - $P(\text{bad}|\text{words}) = 0.67$  (highest)
  - $P(\text{indifferent}|\text{words}) = 0.33$
- **Classification:**
  - "Bad" because  $0.67 \geq 0.33 \geq 0$
  - Even though prior for "good" was highest (0.5)
  - Likelihood updates prior to posterior

**EXAM TRAP:** Classification = **highest posterior probability** - not just prior!

# Q57: Smoothing Effect Example - Tutorial

**QUESTION:** How did Laplace smoothing change the classification in the customer service example?

**OPTIONS:**

- ① It didn't change the classification
- ② It changed "Good" from 0 to 2.9% probability
- ③ It made "Bad" the lowest probability
- ④ It made classification impossible

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Before smoothing:**
  - $P(\text{good}|\text{words}) = 0$
  - $P(\text{bad}|\text{words}) = 0.67$
  - $P(\text{indifferent}|\text{words}) = 0.33$
- **After smoothing ( $\lambda = 1$ ):**
  - $P(\text{good}|\text{words}) = 0.029$  (increased from 0)
  - $P(\text{bad}|\text{words}) = 0.583$  (decreased)
  - $P(\text{indifferent}|\text{words}) = 0.388$  (increased slightly)
- **Classification unchanged:**
  - "Bad" still highest (0.583 > 0.388 > 0.029)
  - But probability of "Good" no longer zero
  - More realistic estimates

**EXAM TRAP:** Smoothing eliminates zero probabilities but may not change classification!

# Q58: Multi-class Classification - Tutorial

**QUESTION:** How does Naïve Bayes handle more than two classes?

**OPTIONS:**

- ❶ It cannot handle more than two classes
- ❷ It computes posterior probability for each class and selects the highest
- ❸ It only uses binary classification
- ❹ It combines classes

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Multi-class extension:**
  - Natural extension of binary case
  - Compute  $P(c|d)$  for each class
  - Select class with highest probability
  - No limitation on number of classes
- **Example (3 classes):**
  - $P(\text{good}|d) = 0.029$
  - $P(\text{bad}|d) = 0.583$
  - $P(\text{indifferent}|d) = 0.388$
  - Select "bad" (highest probability)
- **Applications:**
  - Sentiment: positive, negative, neutral
  - Topic classification: finance, politics, sports, etc.
  - Document categorization

**EXAM TRAP:** Multi-class = choose class with highest posterior!

# Q59: Naïve Bayes Advantages - Tutorial

**QUESTION:** What are the main advantages of Naïve Bayes?

**OPTIONS:**

- ❶ It always has 100% accuracy
- ❷ Simplicity, speed, and solid classification accuracy
- ❸ It works without any assumptions
- ❹ It requires no training data

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Key advantages:**

❶ **Simplicity:**

- Easy to understand
- Easy to implement
- Few parameters to tune

❷ **Speed:**

- Fast training
- Fast prediction
- Scales to large datasets

❸ **Accuracy:**

- Solid classification accuracy
- Works well despite assumptions
- Competitive with more complex models

• **When to use:**

- High-dimensional data
- Text classification
- Quick baseline models

**EXAM TRAP:** Naïve Bayes advantages = **simple, fast, and accurate!**

**QUESTION:** In the appendix problem, what classification is made for the new instance?

**OPTIONS:**

- ① Buy (probability = 0.98)
- ② Sell (probability = 0.02)
- ③ Cannot be classified
- ④ Both equally likely

**ANSWER: A**

**DETAILED TUTORIAL:**

- **Data:**
  - 5 buys, 5 sells
  - Prior:  $P(buy) = 0.5$ ,  $P(sell) = 0.5$
  - New document: words 1, 2, 3 present; words 4, 5 absent
- **Conditional probabilities:**
  - $P(buy) \times \prod P(w_i|buy) = 0.0768$
  - $P(sell) \times \prod P(w_i|sell) = 0.00192$
- **Posterior probabilities:**
  - $P(buy|words) = 0.0768 / (0.0768 + 0.00192) = 0.98$
  - $P(sell|words) = 0.00192 / (0.0768 + 0.00192) = 0.02$
- **Classification:** Buy

**EXAM TRAP:** Posterior normalization = divide by sum of all posteriors!

# Q61: Word Embedding Definition - Tutorial

**QUESTION:** What is word embedding in NLP?

**OPTIONS:**

- ① A method to remove words
- ② A technique to capture word meaning by representing words in a continuous vector space
- ③ A method to count word frequencies
- ④ A technique to stem words

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Word embedding definition:**
  - Represents words as dense vectors
  - Captures semantic meaning
  - Similar words have similar vectors
  - Based on distributional hypothesis
- **Distributional hypothesis:**
  - "You shall know a word by the company it keeps"
  - Similar meanings appear in similar contexts
  - Words appearing together are related
- **Advantages over BoW:**
  - Captures meaning
  - Dense vectors (not sparse)
  - Similar words close in space
  - Better for downstream tasks

**EXAM TRAP:** Word embeddings = capture meaning in dense vectors!

## Q62: Word2Vec Definition - Tutorial

**QUESTION: What is Word2Vec?**

**OPTIONS:**

- ❶ A word stemming algorithm
- ❷ A Google-developed algorithm for generating word embeddings using neural networks
- ❸ A dictionary approach to NLP
- ❹ A pre-processing technique

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Word2Vec definition:**
  - Developed by Google (Mikolov et al., 2013)
  - Neural network-based algorithm
  - Generates word embeddings
  - Reduces dimensionality
- **Two architectures:**
  - ❶ **CBOW (Continuous Bag of Words):**
    - Predict target word from context
    - Faster to train
  - ❷ **Skip-gram:**
    - Predict context words from target word
    - Better for rare words
    - More computationally expensive
- **Capabilities:**
  - Captures semantic relationships
  - "king - man + woman equals queen"
  - Semantic analogies

**EXAM TRAP:** Word2Vec equal [Google algorithm for word embeddings!](#)

# Q63: Context in Word Embeddings - Tutorial

**QUESTION:** How do word embeddings capture context?

**OPTIONS:**

- ❶ By ignoring context
- ❷ By paying attention to positions of words and their surrounding words
- ❸ By counting all words equally
- ❹ By removing all punctuation

**ANSWER: B**

**DETAILED TUTORIAL:**

❶ **Context capture methods:**

❶ **Position information:**

- Track which words appear together
- Window-based co-occurrence
- Similar words appear in similar positions

❷ **Surrounding words:**

- Consider words before and after
- Skip-gram uses surrounding context
- CBOW uses context to predict target

❸ **One-hot encoding limitation:**

- Large and sparse
- No notion of similarity
- Each word independent

❹ **Embedding solution:**

- Dense vectors
- Similar contexts = similar vectors
- Captures semantic meaning

**EXAM TRAP:** Word embeddings capture context by [analyzing surrounding words!](#)



## Q64: Skip-gram vs CBOW - Tutorial

**QUESTION:** What is the difference between Skip-gram and CBOW?

**OPTIONS:**

- ❶ They are the same algorithm
- ❷ Skip-gram predicts context from target; CBOW predicts target from context
- ❸ Skip-gram is for English only
- ❹ CBOW is for English only

**ANSWER: B**

**DETAILED TUTORIAL:**

- **CBOW (Continuous Bag of Words):**

- Input: context words
- Output: target word
- Faster to train
- Better for frequent words
- "Given surrounding words, what's the missing word?"

- **Skip-gram:**

- Input: target word
- Output: context words
- Better for rare words
- More computationally expensive
- "Given a word, what words surround it?"

- **Choice depends on:**

- Data size
- Word frequency distribution
- Computational resources

**EXAM TRAP:** Skip-gram = target  $\rightarrow$  context; CBOW = context  $\rightarrow$  target!

**QUESTION:** What is one-hot encoding in NLP?

**OPTIONS:**

- ❶ A technique to encode temperature
- ❷ Representing words as binary vectors with a 1 in the position of that word and 0 elsewhere
- ❸ A method to count word frequencies
- ❹ A technique to remove stop words

**ANSWER: B**

**DETAILED TUTORIAL:**

- **One-hot encoding:**
  - Vector length = vocabulary size
  - One position = 1 (the word's position)
  - All other positions = 0
  - Example: [0, 0, 1, 0, 0, ...]
- **Limitations:**
  - Very sparse
  - No semantic meaning
  - All words equidistant
  - Cannot capture relationships
- **Comparison with embeddings:**
  - One-hot: Sparse, no meaning
  - Embeddings: Dense, captures meaning

**EXAM TRAP:** One-hot = **binary vector with one 1** - no semantic meaning!

# Q66: Dimensionality Reduction in NLP - Tutorial

**QUESTION:** Why is dimensionality reduction important in NLP?

**OPTIONS:**

- ❶ To make documents longer
- ❷ To reduce sparse, high-dimensional vectors to dense, lower-dimensional representations
- ❸ To add more features
- ❹ To increase computation time

**ANSWER: B**

**DETAILED TUTORIAL:**

- **The problem:**
  - Vocabulary can have 10,000+ words
  - Vectors are very long
  - Very sparse (many zeros)
  - Slow and inefficient
- **Reduction methods:**
  - ❶ **PCA:** Principal Component Analysis
  - ❷ **SVD:** Singular Value Decomposition
  - ❸ **Word2Vec:** Neural network-based
  - ❹ **t-SNE:** For visualization
- **Benefits:**
  - Dense representations
  - Faster computation
  - Better performance
  - Captures relationships

**EXAM TRAP:** Dimensionality reduction = **sparse** → **dense** representations!

# Q67: BoW vs Word Embeddings - Tutorial

**QUESTION:** What is a key advantage of word embeddings over BoW?

**OPTIONS:**

- ① Embeddings are sparser
- ② Embeddings capture semantic meaning and relationships between words
- ③ Embeddings are faster to compute
- ④ Embeddings don't require pre-processing

**ANSWER: B**

**DETAILED TUTORIAL:**

• **BoW limitations:**

- Loses context
- No semantic meaning
- Sparse vectors
- Ignores word relationships

• **Embedding advantages:**

① **Semantic meaning:**

- Similar words close in space
- Captures relationships
- Analogies possible

② **Dense vectors:**

- Lower dimensional
- More efficient
- Better for downstream tasks

③ **Context awareness:**

- Considers surrounding words
- Better representation
- More accurate

**EXAM TRAP:** Embeddings = capture semantic meaning - BoW doesn't!

# Q68: Skip-gram Details - Tutorial

**QUESTION:** What is Skip-gram used for in NLP?

**OPTIONS:**

- ❶ To remove stop words
- ❷ To predict surrounding words given a target word
- ❸ To count word frequencies
- ❹ To stem words

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Skip-gram architecture:**
  - Input: target word
  - Output: context words (surrounding words)
  - Uses neural network
  - Predicts probability of context words
- **Training objective:**
  - Maximize probability of context words
  - Given target word
  - Slide window over text
  - Learn embeddings
- **Window size:**
  - Typically 2-5 words each side
  - Larger window = more global context
  - Smaller window = more syntactic

**EXAM TRAP:** Skip-gram = target word → predict context words!

**QUESTION:** What is a famous semantic relationship captured by Word2Vec?

**OPTIONS:**

- ① "king - man + woman equals queen"
- ② "king + man + woman equals queen"
- ③ "king - queen equals man - woman"
- ④ All words are equally related

**ANSWER: A**

**DETAILED TUTORIAL:**

- **Famous analogy:**
  - $v_{king} - v_{man} + v_{woman} \approx v_{queen}$
  - Captures gender relationship
  - Preserves arithmetic operations
- **Other examples:**
  - "Paris - France + Italy equals Rome"
  - "walking - walk + run equals running"
  - Captures multiple relationships
- **Why it works:**
  - Vector space captures semantic relationships
  - Similar relationships have similar vector offsets
  - Enables analogical reasoning

**EXAM TRAP:** Word2Vec captures **semantic relationships** via vector arithmetic!

**QUESTION:** How are NLP models evaluated?

**OPTIONS:**

- ① Only by accuracy
- ② Using confusion matrices, accuracy, precision, recall, F1 when labels exist
- ③ Only by speed
- ④ Only by data requirements

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Labeled data evaluation:**
  - ① **Confusion Matrix:** TP, TN, FP, FN
  - ② **Accuracy:** Overall correctness
  - ③ **Precision:** Of predicted positives, how many were right?
  - ④ **Recall:** Of actual positives, how many did we catch?
  - ⑤ **F1:** Harmonic mean of precision and recall
- **Unlabeled data evaluation:**
  - Use unsupervised evaluation methods
  - Qualitative assessment
  - Subject matter expert review
- **Other considerations:**
  - Speed of algorithm
  - Data requirements
  - Computational resources

**EXAM TRAP:** Evaluation = **confusion matrix metrics** when labels exist!

**QUESTION:** What metrics are used to evaluate NLP classification models?

**OPTIONS:**

- ❶ Only accuracy
- ❷ Accuracy, precision, recall, and F1 from confusion matrix
- ❸ Only precision
- ❹ Only recall

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Standard evaluation metrics:**

❶ **Accuracy:**

- $(TP + TN) / (TP + TN + FP + FN)$
- Overall correctness

❷ **Precision:**

- $TP / (TP + FP)$
- "Of documents predicted as class X, how many were correct?"

❸ **Recall:**

- $TP / (TP + FN)$
- "Of documents actually in class X, how many did we catch?"

❹ **F1:**

- Harmonic mean of precision and recall
- Balances both metrics

**EXAM TRAP:** Use **multiple metrics** - not just accuracy!



# Q72: Labeled vs Unlabeled Data - Tutorial

**QUESTION:** How is NLP evaluation different for unlabeled data?

**OPTIONS:**

- ❶ No evaluation is possible
- ❷ Use unsupervised evaluation methods or qualitative assessment
- ❸ Use the same metrics as labeled data
- ❹ Evaluation is not needed

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Unlabeled data challenges:**

- No "right answer" to compare against
- Cannot compute confusion matrix
- Need different evaluation approaches

• **Evaluation methods:**

❶ **Subject matter experts:**

- Manually review sample of classifications
- Assess accuracy qualitatively
- Expensive but reliable

❷ **Unsupervised metrics:**

- Cohesion and separation
- Silhouette score
- Domain-specific validation

❸ **Indirect validation:**

- Check if results make sense
- Correlation with known variables
- Predictive power for other tasks

**EXAM TRAP:** Unlabeled data requires **alternative evaluation methods!**

# Q73: Speed and Data Requirements - Tutorial

**QUESTION:** Why should speed and data requirements be considered in NLP evaluation?

**OPTIONS:**

- ❶ They are not important
- ❷ Textual databases can be vast and sparse vectors are hard to handle efficiently
- ❸ Speed is never a concern
- ❹ All NLP models are equally fast

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Practical considerations:**

❶ **Data volume:**

- NLP datasets can be enormous
- Millions of documents
- Billions of words

❷ **Sparsity:**

- Sparse vectors are inefficient
- Slow to process
- Memory intensive

❸ **Model complexity:**

- Neural networks are slow to train
- Require large amounts of data
- May be impractical for some applications

• **Tradeoffs:**

- Accuracy vs speed
- Complexity vs feasibility
- Resource constraints matter

**EXAM TRAP:** NLP can be **computationally expensive** – speed and data matter!

# Q74: BoW Limitations - Tutorial

**QUESTION:** What is a major limitation of simple BoW methods?

**OPTIONS:**

- ❶ They are too fast
- ❷ They are not useful for determining context or intended meaning
- ❸ They are too accurate
- ❹ They require too much data

**ANSWER: B**

**DETAILED TUTORIAL:**

• **BoW limitations:**

❶ **Loss of context:**

- Word order ignored
- Phrase meaning lost
- No understanding of relationships

❷ **No semantic meaning:**

- Words independent
- No concept of similarity
- "good" and "great" treated equally

❸ **Negation issues:**

- "not good" vs "good" both contain "good"
- Sentiment is reversed but BoW doesn't know

• **Advanced solutions:**

- n-grams
- Skip-grams
- Word embeddings
- Contextual embeddings (BERT)

**EXAM TRAP:** BoW fails to capture **context and meaning!**

# Q75: Advanced NLP Techniques - Tutorial

**QUESTION:** What advanced techniques help determine word context?

**OPTIONS:**

- ❶ Only BoW
- ❷ n-grams, Skip-grams, and word embeddings
- ❸ Only stop word removal
- ❹ Only stemming

**ANSWER: B**

**DETAILED TUTORIAL:**

❶ **Context-aware techniques:**

❶ **n-grams:**

- n contiguous words as one unit
- Captures phrase meaning
- Handles negations

❷ **Skip-grams:**

- Predict context from target
- Captures semantic relationships
- Word2Vec uses this

❸ **Word embeddings:**

- Dense vector representations
- Similar words close together
- Captures meaning

❹ **Contextual embeddings:**

- BERT, GPT, etc.
- Context-dependent representations
- State-of-the-art

**EXAM TRAP:** n-grams, Skip-grams, and embeddings capture context!

**QUESTION:** What is the advantage of TF-IDF over simple word counts?

**OPTIONS:**

- ① TF-IDF is faster to compute
- ② TF-IDF assigns higher weights to rare but important words
- ③ TF-IDF counts all words equally
- ④ TF-IDF doesn't require a corpus

**ANSWER: B**

**DETAILED TUTORIAL:**

- **TF-IDF advantages:**
  - ① **Weights rare words higher:**
    - Words appearing in few documents get high IDF
    - More discriminatory power
    - Better for distinguishing documents
  - ② **Weights common words lower:**
    - Words in all documents get low IDF
    - No discriminatory power
    - Reduced noise
- **Comparison:**
  - Simple word count: treats all words equally
  - TF-IDF: distinguishes important from common words
  - Better feature representation

**EXAM TRAP:** TF-IDF = higher weight for rare words!

# Q77: Document Similarity Use - Tutorial

**QUESTION:** How is document similarity measured in NLP?

**OPTIONS:**

- ❶ By comparing document titles
- ❷ By comparing document vectors using similarity metrics (cosine similarity)
- ❸ By counting words in each document
- ❹ By comparing document lengths

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Similarity measurement:**

❶ **Document vectors:**

- Create vectors for each document
- Using BoW, TF-IDF, or embeddings
- Vectors represent document content

❷ **Similarity metrics:**

- Cosine similarity:  $\cos(\theta) = \frac{v_1 \cdot v_2}{||v_1|| \times ||v_2||}$
- Jaccard similarity
- Euclidean distance

• **Applications:**

- Duplicate detection
- Plagiarism detection
- Document clustering
- Information retrieval

**EXAM TRAP:** Document similarity = **comparing document vectors** - often cosine similarity!

## Q78: Cosine Similarity - Tutorial

**QUESTION:** What is cosine similarity used for in NLP?

**OPTIONS:**

- ❶ To count word frequencies
- ❷ To measure similarity between two document vectors
- ❸ To remove stop words
- ❹ To stem words

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Cosine similarity formula:**

- $\cos(\theta) = \frac{v_1 \cdot v_2}{||v_1|| \times ||v_2||}$
- $v_1 \cdot v_2$ : dot product
- $||v||$ : magnitude of vector
- Range: -1 to 1

• **Advantages:**

- Scale-invariant
- Works well with high-dimensional vectors
- Common in text analysis
- Easy to interpret

• **Example:**

- Document vectors are very similar  $\rightarrow$  cosine equals 1
- Document vectors are different  $\rightarrow$  cosine equals 0
- Opposite  $\rightarrow$  cosine equals -1

**EXAM TRAP:** Cosine similarity = measures document vector similarity!

# Q79: Stemming vs Lemmatization Tradeoff - Tutorial

**QUESTION: What is the tradeoff between stemming and lemmatization?**

**OPTIONS:**

- ① No tradeoff - they are the same
- ② Stemming is faster but less accurate; lemmatization is slower but more accurate
- ③ Stemming is more accurate
- ④ Lemmatization is faster

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Stemming:**

• **Advantages:**

- Very fast
- Simple rules
- Low computational cost

• **Disadvantages:**

- Less accurate
- May create non-words
- Loses meaning

• **Lemmatization:**

• **Advantages:**

- More accurate
- Always produces real words
- Preserves meaning

• **Disadvantages:**

- Slower
- More computationally expensive
- Requires dictionary lookup

• **Choice depends on:**

- Speed requirements
- Accuracy requirements
- Application context

**EXAM TRAP:** Stemming = fast but less accurate; Lemmatization = slow but more accurate!



# Q80: NLP Challenges Summary - Tutorial

**QUESTION:** What are the main challenges in NLP?

**OPTIONS:**

- ❶ Only computational speed
- ❷ Ambiguity, context dependence, noise, and sarcasm/detecting tone
- ❸ Only data size
- ❹ Only language barriers

**ANSWER: B**

**DETAILED TUTORIAL:**

• Key challenges:

❶ **Ambiguity:**

- Same words have different meanings
- Polysemy and homonymy
- Context determines meaning

❷ **Context dependence:**

- Meaning depends on surrounding words
- Need to capture relationships
- Complex language structure

❸ **Noise:**

- Spelling errors
- Abbreviations
- Colloquialisms
- Social media language

❹ **Tone detection:**

- Sarcasm hard to detect
- Sentiment nuances
- Emotional content

**EXAM TRAP:** NLP challenges include **ambiguity, context, noise, and tone!**

# Q81: Sentiment Analysis Steps - Tutorial

**QUESTION:** What are the basic steps for sentiment analysis using a dictionary approach?

**OPTIONS:**

- ❶ Only count positive words
- ❷ Establish dictionary, pre-process text, replace with stems/lemmas, calculate proportions
- ❸ Only count negative words
- ❹ Use machine learning only

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Sentiment analysis steps:**

❶ **Establish dictionary:**

- Three lists: positive, negative, neutral
- Loughran-McDonald for financial text
- Pre-defined word lists

❷ **Pre-process text:**

- Remove punctuation, symbols
- Remove stop words
- Convert to lowercase

❸ **Replace with stems/lemmas:**

- Standardize word forms
- Reduce vocabulary
- Treat similar words equally

❹ **Calculate proportions:**

- Proportion positive words
- Proportion negative words
- Proportion neutral words
- Net sentiment = positive - negative

**EXAM TRAP:** Steps = dictionary, pre-process, stem, calculate proportions!

## Q82: BoW Model Explanation - Tutorial

**QUESTION:** Explain how a bag of words model is used in NLP.

**OPTIONS:**

- ❶ It preserves word order
- ❷ It treats each term identically, counts frequencies, stores in vectors for analysis
- ❸ It removes all words
- ❹ It only works for numbers

**ANSWER: B**

**DETAILED TUTORIAL:**

• **BoW process:**

❶ **Treat terms identically:**

- Ignore word order and position
- Each word considered separately
- No context information

❷ **Count frequencies:**

- Count occurrences of each word
- Binary (presence/absence) or count
- Store in vector

❸ **Store in vectors:**

- Each document = one vector
- Vocabulary defines vector dimensions
- Can be analyzed statistically

• **Applications:**

- Dictionary-based sentiment analysis
- Machine learning classification
- Feature extraction for other models

**EXAM TRAP:** BoW = **treats each word identically, counts, stores in vectors!**

**QUESTION:** What are the two assumptions of Naïve Bayes for document grouping?

**OPTIONS:**

- ① Words are ordered and have different weights
- ② Each word is independent of others and each word has equal weighting
- ③ Words are dependent and have different weights
- ④ Words are random

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Assumption 1: Independence:**

- $P(w_1, w_2, \dots, w_n|c) = P(w_1|c) \times P(w_2|c) \times \dots \times P(w_n|c)$
- Each word independent of all others
- Conditional independence given class

• **Assumption 2: Equal weighting:**

- Each word has equal information value
- After removing stop words
- No word is more important than others

• **Why these assumptions:**

- Make the problem tractable
- Work well in practice
- Simplify calculations significantly

**EXAM TRAP:** Naïve Bayes = independence + equal weighting - two key assumptions!

# Q84: Negation Problem and Solution - Tutorial

**QUESTION:** Why do negation words cause problems in document classification and how can they be handled?

**OPTIONS:**

- ❶ They don't cause problems
- ❷ Negations reverse meaning; use n-grams to pair negation with following words
- ❸ Remove all negation words
- ❹ Treat negation as positive

**ANSWER: B**

**DETAILED TUTORIAL:**

- **The negation problem:**
  - Example: "not adequate" vs "adequate"
  - BoW sees "adequate" in both
  - Meaning is opposite
  - Incorrect sentiment classification
- **n-gram solution:**
  - Use bi-grams or higher-order n-grams
  - Treat "not adequate" as one unit
  - "not sufficient" as one unit
  - "isn't good" as one unit
- **Alternative solutions:**
  - Negation detection algorithms
  - Dependency parsing
  - Identify scope of negation
  - Negation word + following word(s)

**EXAM TRAP:** Negation problem = solved with n-grams pairing negation + following word!

# Q85: Zero TF-IDF Explanation - Tutorial

**QUESTION: Why do some words have zero TF-IDF values?**

**OPTIONS:**

- ❶ Because they are misspelled
- ❷ Because they appear in every document and have no discriminatory power
- ❸ Because they are too long
- ❹ Because they are not in the vocabulary

**ANSWER: B**

**DETAILED TUTORIAL:**

• **TF-IDF = 0 condition:**

- $IDF = \ln(D / \sum d_{i,j})$
- If word appears in all D documents:  $\sum d_{i,j} = D$
- $IDF = \ln(D/D) = \ln(1) = 0$
- $TF-IDF = TF \times 0 = 0$

• **Example:**

- "this" appears in all 4 documents in Box 9.2
- Zero discriminatory power
- Cannot help distinguish documents

• **Implication:**

- Common words carry no weight
- Only rare words matter
- Stop words often have zero TF-IDF

**EXAM TRAP:** Zero TF-IDF = word appears in all documents - no discriminatory power!

# Q86: TF-IDF Variation by Document - Tutorial

**QUESTION:** Why does the same word have different TF-IDF scores in different documents?

**OPTIONS:**

- ❶ Because IDF varies by document
- ❷ Because TF varies by document (document length and frequency differences)
- ❸ Because vocabulary changes
- ❹ Because documents have different authors

**ANSWER: B**

**DETAILED TUTORIAL:**

• **TF variation:**

- $TF = w_{i,j} / |v_j|$
- $w_{i,j}$  = frequency of word in document
- $|v_j|$  = total words in document
- Different lengths  $\Rightarrow$  different TFs

• **Example:**

- Word appears 3 times in 100-word document:  $TF = 0.03$
- Word appears 3 times in 50-word document:  $TF = 0.06$
- Same word count, different TF

• **IDF is constant:**

- IDF depends only on corpus
- Same for all documents
- TF varies  $\Rightarrow$  TF-IDF varies

**EXAM TRAP:** TF varies by document (length, frequency)  $\Rightarrow$  **TF-IDF varies by document!**

## Q87: Appendix Problem Unknown Words - Tutorial

**QUESTION:** If new words appear in test documents not in the training sample, how should they be treated?

**OPTIONS:**

- ❶ Add them to the vocabulary
- ❷ Ignore them because they cannot help with classification
- ❸ Remove the entire document
- ❹ Use them as stop words

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Unknown word handling:**
  - Words not in training vocabulary
  - No information about their distribution
  - Cannot calculate  $P(w|c)$
  - Cannot contribute to classification
- **Why ignore them:**
  - They provide no discriminatory information
  - Would cause the model to fail
  - Better to exclude than to guess
- **Possible solution:**
  - Extend labeled sample to include new words
  - Use out-of-vocabulary handling
  - Build larger training vocabulary

**EXAM TRAP:** Unknown words = **ignored** - no information about them!



# Q88: Smoothing Explanation - Tutorial

**QUESTION:** Why is smoothing required in Naïve Bayes applications and how does it work?

**OPTIONS:**

- ❶ It's not required
- ❷ To avoid zero probability problems by adding a positive integer to all counts
- ❸ To remove stop words
- ❹ To stem words

**ANSWER: B**

**DETAILED TUTORIAL:**

• **Why smoothing is needed:**

- Zero probability problem
- Word not in training class  $\rightarrow P = 0$
- Product becomes zero
- Unrealistic and draconian

• **How smoothing works:**

- Add  $\lambda$  to all counts
- $\lambda = 1$ : Laplace smoothing (most common)
- $P(w|c) = \frac{\text{count}(w,c) + \lambda}{\text{total words in } c + \lambda \times |V|}$
- Ensures all probabilities  $> 0$

• **Example:**

- Before:  $0/4 = 0$
- After:  $1/6 = 0.167$
- Small positive probability

**EXAM TRAP:** Smoothing = add to counts to avoid zero probabilities!

# Q89: Laplace Smoothing Impact - Tutorial

**QUESTION:** What happens to the probabilities under Laplace smoothing?

**OPTIONS:**

- ① They become more extreme
- ② They are pushed toward a uniform distribution
- ③ They become negative
- ④ They don't change

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Effect of Laplace smoothing:**
  - Adds counts to all classes
  - All probabilities become positive
  - Shrinks toward uniform distribution
  - Reduces extreme values
- **How it works:**
  - Similar to regularization (Lasso/Ridge)
  - $\lambda$  controls strength of smoothing
  - Larger  $\lambda$  = more smoothing
  - Smaller  $\lambda$  = less smoothing
- **Sample size effect:**
  - Small sample: large impact
  - Large sample: small impact
  - Effect diminishes with more data

**EXAM TRAP:** Laplace smoothing = **shrinks toward uniform distribution!**

**QUESTION:** In the appendix problem, what is the most likely recommendation?

**OPTIONS:**

- ① Buy (probability = 0.98)
- ② Sell (probability = 0.02)
- ③ Hold
- ④ Cannot be determined

**ANSWER: A**

**DETAILED TUTORIAL:**

• **Given data:**

- 5 buys, 5 sells (prior = 0.5 each)
- New document: words 1, 2, 3 present; words 4, 5 absent

• **Conditional probabilities matrix:**

- Word 1:  $P=4/5$  (buy),  $1/5$  (sell)
- Word 2:  $P=5/5$  (buy),  $2/5$  (sell)
- Word 3:  $P=2/5$  (buy),  $3/5$  (sell)
- Word 4:  $P=4/5$  (buy, absent),  $2/5$  (sell, absent)
- Word 5:  $P=3/5$  (buy, absent),  $1/5$  (sell, absent)

• **Calculations:**

- Buy likelihood =  $0.8 \times 1.0 \times 0.4 \times 0.8 \times 0.6 \times 0.5 = 0.0768$
- Sell likelihood =  $0.2 \times 0.4 \times 0.6 \times 0.4 \times 0.2 \times 0.5 = 0.00192$

• **Result:** Buy (98% probability)

**EXAM TRAP:** Work through the full calculation - **buy is much more likely!**

# Q91: NLP Pre-processing Steps - Tutorial

**QUESTION:** What are the main pre-processing steps in NLP?

**OPTIONS:**

- ❶ Only tokenization
- ❷ Tokenization, stop word removal, stemming/lemmatization, and feature extraction
- ❸ Only stemming
- ❹ Only stop word removal

**ANSWER: B**

**DETAILED TUTORIAL:**

❶ **Pre-processing steps:**

❶ **Tokenization:**

- Split into sentences and words
- Remove punctuation
- Convert to lowercase

❷ **Stop word removal:**

- Remove common words with no value
- "the", "a", "and", etc.

❸ **Stemming/Lemmatization:**

- Reduce words to root form
- Standardize vocabulary

❹ **Feature extraction:**

- Convert to numeric vectors
- BoW, TF-IDF, embeddings

**EXAM TRAP:** Pre-processing = tokenization, stop words, stemming/lemmatization, feature extraction!

**QUESTION:** What is the difference between sentence tokenization and word tokenization?

**OPTIONS:**

- ❶ They are the same thing
- ❷ Sentence tokenization splits at punctuation; word tokenization splits into words
- ❸ Sentence tokenization removes words
- ❹ Word tokenization removes sentences

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Sentence tokenization:**
  - Splits document at periods (.)
  - Question marks (?)
  - Exclamation marks (!)
  - Creates sentence boundaries
- **Word tokenization:**
  - Splits sentences into words
  - Removes punctuation
  - Removes spacing and special symbols
  - Converts to lowercase
- **Example:**
  - Document: "Hello world. How are you?"
  - Sentence tokens: ["Hello world.", "How are you?"]
  - Word tokens: ["hello", "world", "how", "are", "you"]

**EXAM TRAP:** Sentence = **splitting at punctuation**; Word = **splitting into words**!

**QUESTION:** Why do stop words vary across applications?

**OPTIONS:**

- ❶ All stop words are the same
- ❷ Words with no value in one context may be useful in another
- ❸ Stop words are always removed
- ❹ Stop words never matter

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Context-dependent stop words:**
  - Some words may be useful in specific domains
  - Example: "bank" is common but important in financial text
  - "stock" is common but important in market analysis
- **Why customization matters:**
  - Standard stop word lists may remove important words
  - Domain-specific vocabulary is important
  - Need to consider the application
- **Examples:**
  - Medical NLP: preserve all medical terms
  - Financial NLP: preserve financial terms
  - Legal NLP: preserve legal terms

**EXAM TRAP:** Stop words are **context-dependent** - not one-size-fits-all!

**QUESTION:** In Figure 9.1, what does the Document Term Matrix show?

**OPTIONS:**

- ❶ Document titles and authors
- ❷ Rows as documents, columns as words, entries as word counts
- ❸ Document dates
- ❹ Document sources

**ANSWER: B**

**DETAILED TUTORIAL:**

- **DTM structure:**
  - **Rows:** Each row = one document
  - **Columns:** Each column = one word from vocabulary
  - **Entries:** Word counts or frequencies
  - Dimensions:  $|W| \times D$
- **Example from Figure 9.1:**
  - Vocabulary: 15 words (high, low, buy, sell, etc.)
  - Documents: d1 and d2
  - d1: high appears twice, stock appears once, etc.
  - d2: bond appears once, volatility appears once, etc.
- **Key features:**
  - Same number of columns for each document
  - Enables quantitative analysis
  - Sparse (many zeros)

**EXAM TRAP:** DTM = documents  $\times$  words matrix with counts!

# Q95: Feature Extraction Purpose - Tutorial

**QUESTION:** Why is feature extraction (text representation) important in NLP?

**OPTIONS:**

- ❶ To make text longer
- ❷ To convert text into numeric vectors for machine learning analysis
- ❸ To remove words
- ❹ To make text more readable

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Feature extraction purpose:**
  - Machine learning models need numeric inputs
  - Text must be converted to numbers
  - Enables statistical analysis
  - Foundation for all ML-based NLP
- **Common methods:**
  - ❶ **Bag of Words:**
    - Simple word counts
    - Binary or count
  - ❷ **TF-IDF:**
    - Weighted word importance
    - Rare words get higher weight
  - ❸ **Word embeddings:**
    - Dense vector representations
    - Captures semantic meaning

**EXAM TRAP:** Feature extraction = text → numeric vectors!



# Q96: Document Term Matrix Dimensions - Tutorial

**QUESTION:** What are the dimensions of a Document Term Matrix?

**OPTIONS:**

- ①  $D \times W$  (documents  $\times$  words)
- ②  $W \times D$  (words  $\times$  documents)
- ③  $D \times D$  (documents  $\times$  documents)
- ④  $W \times W$  (words  $\times$  words)

**ANSWER: A**

**DETAILED TUTORIAL:**

- **DTM dimensions:**
  - Rows:  $D$  = number of documents
  - Columns:  $W$  = number of words in vocabulary
  - Dimensions:  $D \times W$
  - Entry: word count or frequency
- **Example:**
  - 100 documents
  - 10,000 word vocabulary
  - Matrix size:  $100 \times 10,000$
  - 1,000,000 entries
- **Implications:**
  - Very large matrices
  - Sparse (mostly zeros)
  - Need efficient storage

**EXAM TRAP:** DTM dimensions = documents  $\times$  words!

# Q97: BoW Vector Example - Tutorial

**QUESTION:** In the BoW example, why does d1 have a 2 for "high" in the count vector?

**OPTIONS:**

- ❶ Because "high" is the most important word
- ❷ Because "high" appears twice in the document
- ❸ Because "high" is a stop word
- ❹ Because "high" is the first word

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Example text:**
  - d1: "... reaching a high of 1550 ... with a high of 1560"
  - "high" appears twice
  - Count vector records 2
- **Count vs Binary:**
  - **Count:** 2 (frequency)
  - **Binary:** 1 (presence)
  - Count gives more information
- **Other words:**
  - "stock" appears once → 1
  - "profit" appears once → 1
  - "rise" appears once → 1
  - "fall" appears once → 1

**EXAM TRAP:** Count BoW = **actual frequency** - not just presence!

**QUESTION:** What is the Bag of n-grams (BoN) approach?

**OPTIONS:**

- ❶ A method to count single words
- ❷ Applying BoW to n-grams (n contiguous words as units)
- ❸ A method to remove n-grams
- ❹ A method to count only bigrams

**ANSWER: B**

**DETAILED TUTORIAL:**

• **BoN definition:**

- Extends BoW to n-grams
- n contiguous words = one unit
- Example: "credit spreads" as one unit
- Handles phrases and negations

• **Example:**

- Document: "The bank is not profitable"
- Bi-grams: ["The bank", "bank is", "is not", "not profitable"]
- "not profitable" as one unit

• **Advantages:**

- Captures phrase meaning
- Handles negations
- Better context information

**EXAM TRAP:** BoN = BoW applied to n-grams - captures phrases!

# Q99: Cosine Similarity Application - Tutorial

**QUESTION:** In document similarity, what does a cosine similarity close to 1 indicate?

**OPTIONS:**

- ❶ Documents are very different
- ❷ Documents are very similar (vectors point in same direction)
- ❸ Documents are opposite
- ❹ Documents are unrelated

**ANSWER: B**

**DETAILED TUTORIAL:**

- **Cosine similarity interpretation:**
  - **cos equals 1:** Very similar documents
  - **cos equals 0:** Unrelated documents
  - **cos equals -1:** Opposite documents
  - Range:  $[-1, 1]$
- **Why it works:**
  - Vectors point in same direction
  - Similar word distributions
  - Angle between vectors is small
  - Magnitude doesn't matter (scale-invariant)
- **Applications:**
  - Finding duplicate documents
  - Information retrieval
  - Document clustering
  - Plagiarism detection

**EXAM TRAP:** cos equals 1 = **very similar documents!**

# Q100: NLP Chapter Summary - Tutorial

**QUESTION:** What are the key takeaways from the NLP chapter?

**OPTIONS:**

- ❶ Only BoW is used in NLP
- ❷ NLP involves pre-processing, feature extraction, dictionary or ML approaches, and careful evaluation
- ❸ Only machine learning is used
- ❹ NLP doesn't require pre-processing

**ANSWER: B**

**DETAILED TUTORIAL:**

• Key takeaways:

❶ **Pre-processing:**

- Tokenization
- Stop word removal
- Stemming/lemmatization
- Feature extraction

❷ **Feature extraction:**

- Bag of Words (BoW)
- n-grams
- TF-IDF weighting

❸ **Approaches:**

- Dictionary-based (heuristic)
- Machine learning (Naïve Bayes, SVM, neural networks)
- Word embeddings

❹ **Evaluation:**

- Confusion matrix metrics
- Speed and data requirements
- Context-aware techniques

**EXAM TRAP:** NLP = pre-processing + feature extraction + modeling + evaluation!



# Summary: Complete NLP Review

## Pre-processing:

- **Tokenization:** Sentences  $\rightarrow$  words
- **Stop Words:** Remove common words with no value
- **Stemming:** Cut prefixes/suffixes (fast, less accurate)
- **Lemmatization:** Dictionary form (slow, more accurate)

## Feature Extraction:

- **BoW:** Binary (presence) or Count (frequency)
- **n-grams:** Contiguous word sequences
- **TF-IDF:**  $TF \times IDF$  - weights rare words higher
- **Embeddings:** Dense vectors capturing meaning (Word2Vec)

## Classification:

- **Dictionary:** Pre-defined word lists (sentiment analysis)
- **Naïve Bayes:** Simple, fast, generative classifier
- **Smoothing:** Laplace smoothing to avoid zero probabilities

## Evaluation:

- **Confusion Matrix:** TP, TN, FP, FN
- **Metrics:** Accuracy, Precision, Recall, F1
- **Challenges:** Ambiguity, context, negation, sarcasm

**Exam Success:** Understand the process from raw text to classification and evaluation!